



Universidad Europea

UNIVERSIDAD EUROPEA DE MADRID

Aprendizaje Automático

Trabajo Grupal:

Proyecto Final

Integrantes:

Martin Eduardo Otero Di Lorenzo

Iker Arredondo Molina

Fernando Sanchez

Wenjie Jiang

CURSO 2025-2026

Contexto: El Problema y el Uso de Imágenes Satelitales

El panorama geográfico en muchos países europeos, notablemente en España, se caracteriza por un contraste acentuado entre los grandes centros urbanos dinámicos y la vasta red de pueblos pequeños y comunidades rurales que enfrentan serios desafíos. Estos desafíos se resumen en un proceso de despoblación persistente, donde la falta de oportunidades económicas y la desconexión de las grandes redes de infraestructura provocan un declive demográfico y social progresivo. Este desequilibrio requiere una capacidad de diagnóstico y monitoreo que supere los métodos estadísticos tradicionales, que a menudo son lentos y no cubren la granularidad necesaria.

En este contexto, la Observación de la Tierra a través de la tecnología satelital se presenta como una herramienta indispensable. Al combinar información sobre la expansión física de las ciudades (obtenida de satélites como Sentinel-2) con datos de Luminosidad Nocturna (provenientes de VIIRS/DMSP), podemos medir la actividad económica y el pulso demográfico de cualquier región, independientemente de su tamaño. La caída de la luminosidad en áreas rurales es un indicador temprano y objetivo del declive, lo que nos permite actuar con mayor celeridad y precisión que con los indicadores tradicionales.

Introducción



G E O L U M I C A
C I T I E S F U T U R E

El proyecto GeoLumica, nace para abordar esta disparidad regional. Su propósito es generar inteligencia geoespacial avanzada para mapear y comprender la compleja dinámica del crecimiento urbano y el riesgo de despoblación en Europa, centrándose específicamente en la relación entre la actividad económica (luminosidad) y la conectividad ferroviaria. Desde el inicio, GeoLumica se concibió con el objetivo general de establecer un sistema de monitoreo continuo que rastree y cuantifique la expansión urbana y la actividad económica en los municipios europeos. Paralelamente, la iniciativa se enfoca en identificar y clasificar las regiones según su potencial de crecimiento o su vulnerabilidad económica y demográfica, y en cuantificar la influencia de infraestructuras críticas como la red ferroviaria en la vitalidad económica de su entorno inmediato. El proyecto busca ser una herramienta de análisis predictivo que permita a las instituciones y empresas tomar decisiones informadas sobre planificación e inversión territorial.

El producto final de GeoLumica es un Mapa Interactivo Web Dinámico. Esta herramienta permitirá a los usuarios visualizar de forma intuitiva las clasificaciones de las zonas, identificar *hotspots* de inversión y áreas de alerta. Si bien el producto final es la visualización, el foco del desarrollo y el presente reporte se centra en la parte fundamental del proyecto: la extracción, el tratamiento, la limpieza y el almacenamiento de las vastas cantidades de datos satelitales y de *Machine Learning* requeridos para alimentar esta plataforma con información robusta y científicamente validada.

Sprint 0: Primeros Pasos

El Sprint 0 se ejecutó para establecer las bases estratégicas de GeoLumica, priorizando la claridad conceptual y la definición de su alcance. La fase se centró en la creación de los materiales de presentación clave, como el Elevator Pitch y el póster-resumen, que sirvieron para consensuar la motivación central del proyecto: transformar datos geoespaciales complejos en inteligencia estratégica para combatir el desequilibrio entre el crecimiento urbano y el declive de las zonas rurales. Se estableció que la relevancia de GeoLumica radica en su capacidad de anticiparse a las tendencias de despoblación y crecimiento mediante la combinación de imágenes ópticas de satélites (expansión urbana), datos de Luminosidad Nocturna (actividad económica) y la red ferroviaria (factor de conectividad).

Además, se definió que, aunque el proyecto tiene un alcance europeo, la fase piloto se centraría en España, y, crucialmente, el análisis debía ser llevado a cabo a nivel municipal en lugar de provincial. Esta granularidad es fundamental para reflejar con precisión la problemática de los pueblos pequeños y asegurar que el producto final, un Mapa Interactivo Web Dinámico, sea útil para las decisiones de inversión y planificación territorial.

La segunda parte del *sprint* se dedicó a identificar a los usuarios clave y a detallar la aplicabilidad del producto. Se determinó que, si bien el mapa interactivo sería accesible a usuarios comunes, el valor estratégico de GeoLumica está intrínsecamente ligado a entidades gubernamentales, operadores de transporte e instituciones financieras. Estos *stakeholders* se beneficiarían directamente de la clasificación de zonas en "en auge" o "en riesgo de despoblación", información vital para la optimización de infraestructuras y la inversión en renta.



Ilustración 1-Flyer Geolumica

Sprint 1: Identificación de Fuentes de Datos y Estrategia

En este primer sprint, bajamos las expectativas iniciales para ver de dónde íbamos a sacar toda la información que necesitábamos. Investigando por internet, nos dimos cuenta de que bajar imágenes de satélite no es tan simple como descargar un archivo cualquiera; cada satélite tiene sus propias APIs y métodos de acceso dependiendo de lo que busques. Al final, decidimos que Sentinel-2 sería nuestra fuente principal para ver el crecimiento de las ciudades, básicamente porque es el que ofrece mayor nitidez y precisión para observar el terreno. Un punto clave fue configurar filtros específicos para el mapa de España: no nos servía cualquier foto, así que tuvimos que aprender a descartar automáticamente todas las imágenes que estuvieran nubladas o borrosas para no ensuciar el análisis.

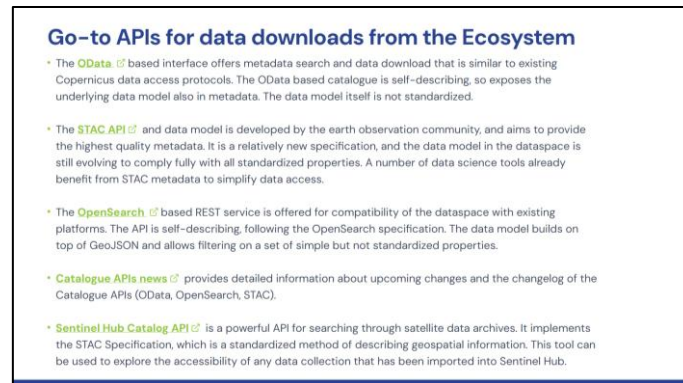


Ilustración 2-Algunas de las diferentes APIs del Ecosistema Copernicus, fuente: <https://dataspace.copernicus.eu/analyse/apis/catalogue-apis>

En paralelo a la parte visual, investigamos las métricas numéricas esenciales para medir la vitalidad económica y la conectividad. Identificamos el VIIRS (Visible Infrared Imaging Radiometer Suite) como la fuente primordial para los datos de luminosidad nocturna; este sensor es capaz de detectar la luminosidad incluso a nivel nocturno, lo que nos permite cuantificar la contaminación lumínica como un factor directo de la actividad humana y el consumo energético.

demás de la luminosidad, definimos la necesidad de integrar variables complementarias como los datos demográficos históricos y la infraestructura de tráfico ferroviario. Este proceso de definición de variables y formatos de interés nos llevó a diseñar una estrategia de control de APIs, Union de datos y almacenamiento, , capaz de gestionar tanto los pesados archivos ráster de los satélites como las bases de datos numéricas, sentando las bases para el tratamiento masivo de datos que vendría en las etapas posteriores.

Dedicamos la parte final del sprint a realizar las primeras pruebas de conexión con las APIs oficiales de Copernicus. Logramos descargar metadatos de los satélites Sentinel-1 y Sentinel-2, parametrizando las búsquedas para que se centraran en el territorio español. Aunque en ese momento todavía no sabíamos cómo usar esos datos o cómo conectaríamos todos esos archivos con las imágenes finales, estas descargas iniciales nos sirvieron para validar que los filtros de búsqueda funcionaban correctamente.

También pudimos iniciar las pruebas con la API OSM (OpenStreetMap) Una API, muy extensa e interesante ya que tenía datos sobre distintos tipos de conexiones en cuanto a transporte en todo el mundo. En general, es una API hecha por la comunidad en base a imágenes satelitales, GPS, y otras fuentes que hacen que sea una herramienta muy interesante.

OpenStreetMap proporciona datos de mapas para miles de sitios web, aplicaciones móviles y dispositivos de hardware

OpenStreetMap lo crea una gran comunidad de mapeadores que aportan y mantienen datos sobre caminos, senderos, cafeterías, estaciones de ferrocarril y muchas cosas más a lo largo de todo el mundo.

Ilustración 3- OSM informacion de la web, fuente: <https://www.openstreetmap.org/#map=6/40.01/-2.49>

Sprint 2: Descargas Iniciales y Estrategia

Al inicio de este segundo sprint, nuestro objetivo principal era consolidar la descarga completa de todas las fuentes de datos previstas. Sin embargo, nos enfrentamos a una realidad más compleja de lo anticipado. Uno de los mayores obstáculos fue la integración en un script main común. Al intentar unificar las llamadas de todos los módulos, detectamos errores críticos derivados de la saturación de las APIs; descubrimos que cada servicio requiere tiempos de descanso específicos para evitar bloqueos por exceso de peticiones. Además, la ejecución de procesos pesados en paralelo provocó que el sistema intentara gestionar múltiples descargas masivas simultáneamente, lo que derivó en problemas de falta de memoria RAM y una ralentización extrema. Para solucionar esto, hemos comenzado a implementar controles de excepciones y gestión de timeouts, aunque algunas de estas tareas de optimización han tenido que ser desplazadas al siguiente sprint para asegurar su correcta ejecución.

Tras múltiples pruebas fallidas con las APIs de OpenStreetMap (OSM) y los datos de luminosidad, estuvimos a punto de rendirnos debido a la dificultad de obtener datos limpios. Sin embargo, decidimos cambiar el enfoque y utilizar un archivo *'municipios_es.geojson'* como base estructural. Este cambio fue fundamental, ya que nos permitió dividir el procesamiento de datos por bloques a nivel municipal en toda España. Gracias a esta base, hemos desarrollado un sistema que transforma las estaciones ferroviarias descargadas en un conjunto de indicadores de accesibilidad muy detallados. No solo contamos cuántas estaciones hay en cada pueblo, sino que calculamos su densidad por kilómetro cuadrado y medimos la distancia mínima y media desde el centro de cada municipio a las estaciones más cercanas.

Para capturar la accesibilidad real, implementamos un sistema de radios de influencia de 1 km y 5 km. Esto es vital porque nos permite identificar si un municipio tiene acceso al tren, aunque la estación esté físicamente justo fuera de su límite administrativo. Además, analizamos la calidad del servicio revisando si las estaciones tienen un operador asignado o si son accesibles para personas con movilidad reducida. Con todas estas variables, logramos clasificar cada municipio en categorías de conectividad: directo, funcional, periférico o aislado. Esta métrica combinada nos ofrece, por primera vez, una visión completa de cómo se distribuye la infraestructura y qué nivel de acceso real tiene cada rincón de España. Abajo algunas de las variables mas importantes.

<i>Nombre de la columna</i>	<i>Descripción</i>	<i>Notas / Relevancia</i>
<i>SOG_ID / LAU_ID</i>	Identificador administrativo y código oficial del municipio.	Clave primaria para unir con otros datos (Joins).
<i>stations_count</i>	Número de estaciones dentro del polígono municipal.	Indicador básico de presencia ferroviaria.
<i>stations_density_km2</i>	Densidad de estaciones por cada km ² .	Permite comparar municipios de distintos tamaños.
<i>stations_in_muni_plus_1km_count</i>	Estaciones dentro del polígono ampliado por 1 km.	Crítico: Evita "falsos ceros" en estaciones limítrofes.
<i>min_distance_km_to_station</i>	Distancia mínima desde el centro a la estación más cercana.	Mide el aislamiento real del núcleo urbano.
<i>accessible_share</i>	Proporción de estaciones con accesibilidad (wheelchair).	Calidad del servicio para personas con movilidad reducida.
<i>category_connectivity</i>	Clasificación final de conectividad del municipio.	Resultado: Clasifica en directo, funcional, aislado o periférico.

Para hablar más en detalle del proceso de descarga de VIIRS, es importante destacar que nos enfrentamos al reto de que la intensidad de luz nocturna por provincia o municipio en España no existe en ningún CSV oficial. La única forma de conseguir esta información era conectándonos a Google Earth Engine, donde la NOAA publica las imágenes satelitales crudas; por lo tanto, nuestro módulo no se limita a leer datos, sino que los fabrica a partir de imágenes. Este enfoque con Earth Engine fue el origen de varios problemas iniciales, especialmente con la autenticación y la gestión de credenciales, lo cual resolvimos asegurando los archivos de acceso mediante el .gitignore para no comprometer la seguridad del repositorio. Además, para poder usar los servicios de Google se necesita una autenticación cada vez que se ejecuta o sino te bloquean la “Access Key”, algo frustrante que sucedió mientras hacíamos pruebas.

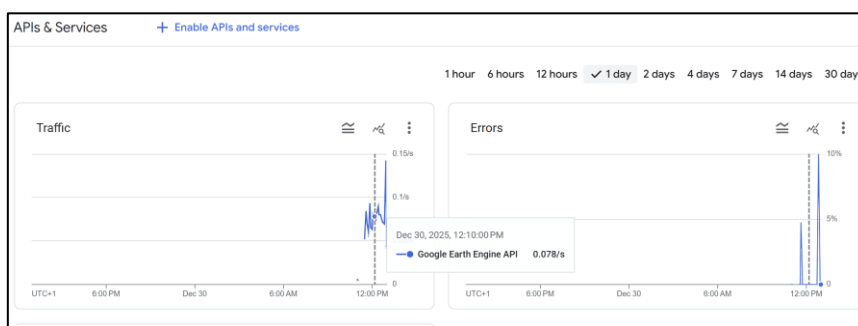


Ilustración 4- Uso de los servicios de Google Earth Engine en la interfaz de Google Cloud

El flujo de trabajo que implementamos para esta fabricación de datos comienza construyendo las geometrías de las provincias españolas a partir del mapa mundial GAUL, filtrando el territorio nacional y fusionando los fragmentos para obtener un polígono único y preciso por cada provincia. Una vez definida la forma geográfica, el código realiza el trabajo mensual de procesar cada imagen VIIRS, recortándola con la forma de la provincia y midiendo la luz para calcular métricas como la media, el mínimo, el máximo y la desviación estándar. En la Ilustración 5 de abajo se muestra un ejemplo del script siendo ejecutado en la consola, donde se puede filtrar por layers por si se quiere marcar más las zonas oscuras

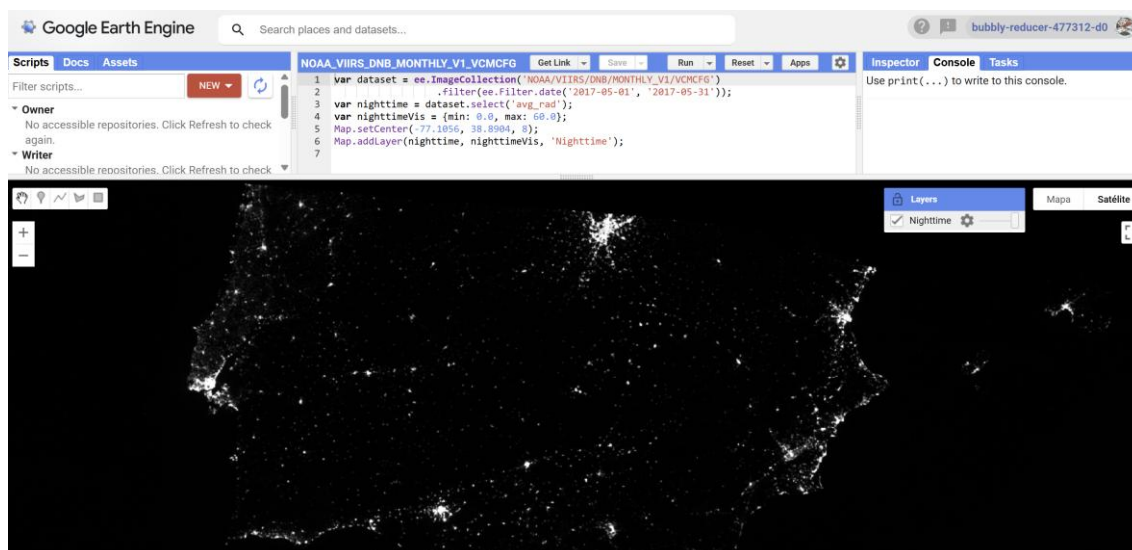


Ilustración 5- Ejemplo de Google Earth Engine con el Dataset usado de Viirs

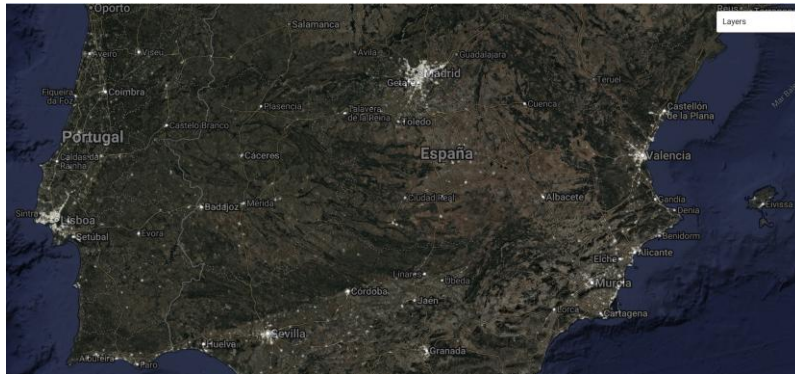


Ilustración 6-Ejemplo de Google Earth Engine con Layer mas temprano

Tras obtener estos valores, utilizamos herramientas para convertir la información de Earth Engine en tablas de datos manejables, aplicando una capa de normalización de nombres de columnas para que el formato fuera siempre consistente. Al finalizar el proceso de todos los meses y años, el sistema concatena los resultados en un archivo CSV final consolidado, el cual queda correctamente ubicado en nuestro sistema de carpetas gracias a una integración en el `main.py`, dejando un flujo de datos histórico y ordenado para el resto de GeoLumica.

Más allá de lo planeado originalmente para el sprint, este sprint sirvió para terminar deficiencias de la fase anterior. Identificamos que los datos poblacionales iniciales carecían de profundidad histórica, por lo que tuvimos que buscar nuevas fuentes que permitieran ver la evolución demográfica real. Asimismo, corregimos el sistema de descarga de Sentinel-2, que anteriormente solo obtenía muestras aisladas; ahora hemos sentido las bases para una descarga masiva, requisito indispensable para el entrenamiento efectivo de nuestros modelos de Machine Learning. Aun así falta comprobar de que todas las imágenes sean realmente útiles, ya que muchas vienen de por sí con cortes geométricos, neblina, sin filtros aplicados, lo cual hace que su utilidad disminuya e inclusive empeore a la hora de entrenar.

Finalmente, realizamos una investigación técnica sobre el formato de las imágenes. Descubrimos que las descargas nativas se realizan en formato .jp2 (JPEG 2000), el cual, aunque eficiente para almacenamiento, no es el más manejable para todas nuestras herramientas de análisis. Por ello, definimos que se necesita un protocolo necesario para convertirlos a formatos más estándar que se usen actualmente. Al mismo tiempo, iniciamos las tareas de limpieza y preprocesamiento básico de los CSV obtenidos de la API de OSM, asegurando que la información de conectividad ferroviaria y puntos de interés esté lista para ser integrada en nuestro mapa interactivo.

Aun así, debido a la complejidad para conseguir datos, tuvimos que aplazar estas cuestiones sobre la descarga de imágenes para el siguiente Sprint. Lo mismo con terminar la limpieza y preprocesamiento de datos.

Sprint 3: Limpieza y Normalizacion

Durante este tercer sprint, hemos logrado finalmente concluir las tareas de limpieza de datos y consolidar un script main común que integra todo el flujo de trabajo del proyecto. Un paso fundamental en este avance ha sido la unificación de los distintos conjuntos de datos a nivel de provincia y municipio, utilizando como eje central el mapeo a partir del archivo '*municipios_es.geojson*' y códigos numéricos. Gracias a esta estrategia, hemos conseguido que todos los datos de un mismo tipo estén unificados y agrupados por año y provincia, lo que resulta extremadamente útil para realizar consultas (queries) eficientes en el futuro filtrando por parámetros específicos. Esta arquitectura se ha aplicado de forma consistente tanto a los datos demográficos como a los obtenidos mediante VIIRS y OSM, garantizando una base de datos coherente.

	A	B	C	D	E	F	G	H
1	region_c...	region_n...	year	provincia_population	municipio	Total	Homb...	Mujer...
2	01	Araba/Ál...	1996	281821.0	Agurain/Salvatierra	3796.0	1929.0	1867.0
3	01	Araba/Ál...	1996	281821.0	Alegría-Dulantzi	1234.0	640.0	594.0
4	01	Araba/Ál...	1996	281821.0	Amurrio	9758.0	4866.0	4892.0
5	01	Araba/Ál...	1996	281821.0	Aramaio	1345.0	704.0	641.0
6	01	Araba/Ál...	1996	281821.0	Armiñón	139.0	73.0	66.0
7	01	Araba/Ál...	1996	281821.0	Arraia-Maeztu	684.0	363.0	321.0
8	01	Araba/Ál...	1996	281821.0	Arratzua-Ubarrundia	745.0	388.0	357.0

Ilustración 7-Poblacion por Provincia y Municipio, Agrupado por año

Para asegurar la conexión entre los distintos archivos CSV y los diferentes tipos de información, llevamos a cabo una normalización exhaustiva de los nombres de provincias y municipios. Detectamos que, por problemas de codificación (encoding), muchos nombres no se representaban con corrección, especialmente aquellos que contenían acentos o la letra "ñ". Para solventar esto, implementamos una función denominada *fix_mojibake*, que nos permitió corregir errores de visualización. Por ejemplo, transformamos registros que aparecían erróneamente como " A CoruÃ±a " en el nombre correcto "La Coruña", o también "A Coruña " en por el gallego. Además, mantuvimos una estructura de códigos de identificación únicos pensando estratégicamente en la futura implementación de nuestra base de datos (BBDD).

01046	Erriberagoitia/Ri...	01	Álava
01046	Erriberagoitia/Ri...	01	Álava
01046	Erriberagoitia/Ri...	01	Álava
01046	Erriberagoitia/Ri...	01	Álava
01047	Ribera Baja/Erri...	01	Álava
01047	Ribera Baja/Erri...	01	Álava
01047	Ribera Baja/Erri...	01	Álava
01047	Ribera Baja/Erri...	01	Álava

Ilustración 8-Ilustracion del Dataset renta media con Valores con "/" para representar lo mismo

En cuanto a la arquitectura del software, aprovechamos este sprint para depurar errores y estructurar de manera más sólida el script main, logrando que tanto la fase de descarga como la de limpieza funcionen bajo un mismo paraguas unificado. En este proceso, realizamos un intento de Dockerización de estas partes del sistema. Sin embargo, nos encontramos con un obstáculo técnico importante, la autenticación de Google Earth Engine se realiza mediante una función que redirige automáticamente al navegador, lo cual no funcionaba correctamente dentro de un contenedor Docker basado en archivos de credenciales estáticos. Aquí es cuando probamos de vuelta utilizar credenciales estáticas, esta vez en un .json, pero termino bloqueándonos el acceso.

Type	Status	Disable reason	Key	Creation date	Expiration date
	 Disabled	Exposed 	313cbf4ec43387a9d0079bdee6f75cb090ab775e	Nov 10, 2025	Jan 1, 10000 

Ilustración 9- Status de la Access Key Bloqueada

A pesar de este contratiempo, logramos mejorar sustancialmente la estructura de carpetas del proyecto, haciéndola mucho más prolija, lógica y fácil de navegar para nosotros y para los Dockerfiles. Esto nos obliga a controlar mejor las importaciones y mejorar el archivo de requisitos el cual lo habíamos dejado de lado y separado por cada subproceso, algo que termino dándonos más problemas.

```
-- Tabla de métricas ferroviarias municipales (OSM)
DROP TABLE IF EXISTS `osm_municipios_metrics`;
CREATE TABLE `osm_municipios_metrics` (
  `PROV_NAME` VARCHAR(100) NULL, -- nombre oficial de la provincia
  `LAU_ID` VARCHAR(50) NOT NULL, -- id canónico unidad administrativa (LAU/UA)
  `LAU_NAME` VARCHAR(255) NULL, -- nombre normalizado del municipio
  `AREA_KM2` DECIMAL(12,3) NULL, -- superficie municipal en km²
  `POP_2023` INT NULL, -- población estimada 2023
  `stations_count` INT NULL, -- estaciones ferroviarias dentro del municipio
  `stations_unique` INT NULL, -- estaciones únicas tras deduplicación
  `stations_density_km2` DECIMAL(8,3) NULL, -- densidad de estaciones por km²
  `stations_with_operator_share` DECIMAL(5,3) NULL, -- proporción con operador definido
  `operator_count` INT NULL, -- estaciones con operador
  `stations_per_10k_pop` DECIMAL(8,3) NULL, -- estaciones por cada 10 000 habitantes
  `stations_within_1km_count` INT NULL, -- estaciones dentro de 1 km del centroid
  `stations_within_5km_count` INT NULL, -- estaciones dentro de 5 km del centroid
  `stations_in_muni_plus_1km_count` INT NULL, -- estaciones en polígono ampliado 1 km
  `stations_in_muni_plus_5km_count` INT NULL, -- estaciones en polígono ampliado 5 km
  `min_distance_km_to_station` DECIMAL(8,3) NULL, -- distancia mínima al centroid (km)
  `mean_distance_km_to_station` DECIMAL(8,3) NULL, -- distancia media a estaciones cercanas (km)
)
```

Ilustración 10-Ejemplo de Tabla OSM con Columnas explicadas anteriormente

Una vez limpios todos los CSVs, procedimos a la creación de la base de datos, trabajando inicialmente en un entorno local con MySQL para probar la conexión y los tiempos de carga debido al gran volumen de información que manejamos. Este proceso resultó ser mucho más sencillo de lo previsto gracias a que habíamos planeado con antelación la unificación de archivos mediante claves y nombres comunes. Aun así, hemos decidido dejar una revisión menor para el siguiente sprint, ya que nuestro objetivo es ampliar el alcance del proyecto incluyendo nuevas fuentes de datos, como la renta media y el consumo eléctrico. Estos nuevos indicadores servirán para complementar las regresiones futuras, diferenciándose de los datos actuales que están más enfocados en los modelos de clustering por provincia y municipio.

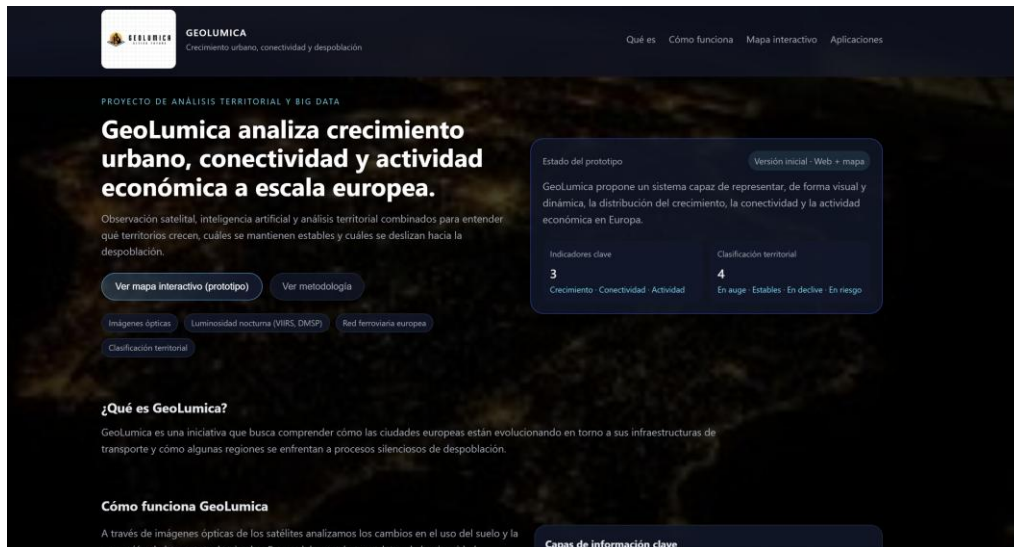


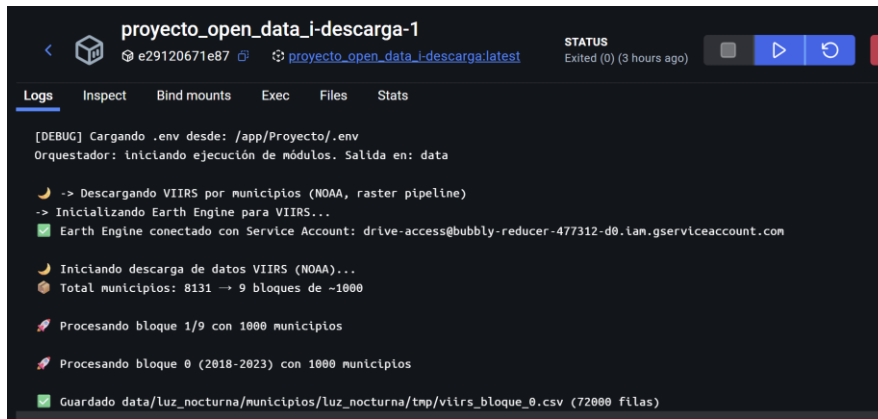
Ilustración 11-Fragmento del Mockup de la pagina web

Finalmente, para cerrar este sprint, desarrollamos un mockup detallado del producto final que presentaremos ante la clase, permitiéndonos visualizar y validar la dirección que tomará GeoLumica en su fase final. Este Mockup incluyo como se quería ver la pagina web, con un espacio al final para el mapa interactivo. Aun asi, no se detallo en las diferentes funciones por usuario, sino mas en como se va a conectar con todo el progreso dado.

Sprint 4: Presentacion del Producto y Docker

En este cuarto sprint, nuestro principal objetivo era cerrar la Dockerización de las tres secciones clave del proyecto: descarga, limpieza y conexión con la base de datos. Sin embargo, el camino estuvo lleno de obstáculos técnicos imprevistos. Uno de los problemas más críticos fue el fallo en las descargas de datos demográficos basados en la API del INE, que empezaron a devolver errores 404 de forma sistemática. Tras una revisión exhaustiva para descartar factores externos como la saturación por fechas (diciembre) o el estado general de la web, confirmamos que el error se debía a una reestructuración interna del INE, que había movido o eliminado las ubicaciones de las tablas que consultábamos. Este contratiempo nos obligó a reajustar nuestras peticiones para asegurar la continuidad del flujo de datos históricos.

Respecto al avance con Docker, logramos resolver con éxito el problema de las credenciales de Google Earth Engine que arrastrábamos desde el sprint anterior. Optamos por mantener el formato .json para las credenciales, lo que ha permitido que el módulo funcione correctamente y sea totalmente portable, pudiendo ejecutarse ahora desde cualquier ordenador sin conflictos de acceso.



```

proyecto_open_data_i-descarga-1
STATUS
Exited (0) (3 hours ago)

Logs Inspect Bind mounts Exec Files Stats

[DEBUG] Cargando .env desde: /app/Proyecto/.env
Orquestador: iniciando ejecución de módulos. Salida en: data

-> Descargando VIIRS por municipios (NOAA, raster pipeline)
-> Inicializando Earth Engine para VIIRS...
✓ Earth Engine conectado con Service Account: drive-access@bubbly-reducer-477312-d0.iam.gserviceaccount.com

Iniciando descarga de datos VIIRS (NOAA)...
Total municipios: 8131 -> 9 bloques de ~1000

Procesando bloque 1/9 con 1000 municipios
Procesando bloque 0 (2018-2023) con 1000 municipios
Guardado data/luz_nocturna/municipios/luz_nocturna/tmp/viirs_bloque_0.csv (72000 filas)
  
```

Ilustración 12-Fragmento de descarga datos VIIRS con Google Earth Engine

Durante este proceso, nos enfrentamos a dificultades con la gestión de dependencias; debido al gran tamaño y la sensibilidad de ciertas librerías, tuvimos que crear un archivo constraints.txt adicional al requirements.txt. Esto fue necesario para fijar versiones específicas de las librerías de Google, evitando errores de compatibilidad que surgían al descargar versiones más recientes o incompletas. Gracias a esto, hemos dado por finalizados los contenedores de descarga y limpieza, dejando únicamente la parte de la BBDD para el cierre final.

Paralelamente a la parte técnica, comenzamos la redacción de esta memoria final del proyecto, documentando detalladamente cada elección, los procesos implementados y, sobre todo, cómo hemos ido resolviendo los problemas surgidos en cada fase. Durante este sprint también realizamos la presentación de nuestro mockup ante la clase. El feedback recibido y la observación de los proyectos de nuestros compañeros nos dieron una perspectiva valiosa: nos dimos cuenta de que necesitábamos "productivizar" aún más la parte de la automatización para que el sistema sea más robusto y autónomo. Esta visión nos reafirmó en nuestra idea inicial, especialmente al ver que nuestro volumen de datos es sólido y competitivo en comparación con otros enfoques.



INEbase

Resultados por municipios, distritos y secciones censales por provincias

Madrid

Indicadores de renta media y mediana

Unidades: €, %

Selección de valores a consultar

Unidades territoriales

Indicadores de renta media y mediana

Periodo

Unidades territoriales	Indicadores de renta media y mediana	Periodo
Municipios(179)	Renta neta media por persona	2023
Distritos(246)	Renta neta media por hogar	2022
Secciones(4541)	Media de la renta por unidad de consumo	2021
28001 Acebeda, La	Mediana de la renta por unidad de consumo	2020
2800101 Acebeda, La distrito 01	Renta bruta media por persona	2019
28002 Alajó	Renta bruta media por hogar	2018
28003 Alameda del Valle		2017
28004 Alamo, El		2016
28005 Alcalá de Henares		2015
28006 Alcobendas		
28007 Alcorcón		
28008 Aldea del Fresno		
Seleccionar/Deseleccionar sub-elementos		

Seleccionados: 1 Total: 6 Seleccionados: 1 Total: 9

Ilustración 13-Ejemplo de Tabla consultada del INE

Para fortalecer aún más nuestras futuras regresiones y análisis, en este sprint hemos agregado nuevas fuentes de datos de alta calidad provenientes del INE, como la Renta Media, el Consumo Eléctrico y datos sobre empresas de transporte en España, desglosados por provincias y municipios. Aunque se pudiese hacer mas específico decidimos dejarlo por municipio para que combine correctamente con el resto de datos.

Al ser fuentes oficiales e históricas, estos datos complementan perfectamente los indicadores de luminosidad y conectividad que ya teníamos, permitiéndonos realizar análisis mucho más profundos que el simple clustering. Con esta infraestructura de datos ya consolidada y los contenedores casi listos, entramos en la recta final del proyecto con una base muy potente.

Sprint 5: Entrega Final y BBDD Terminada

En esta etapa final del proyecto, hemos centrado nuestros esfuerzos en garantizar que todo el trabajo realizado sea portable y ejecutable en cualquier entorno, especialmente en el ordenador de la universidad (L.O.R.C.A.). Por este motivo, decidimos realizar un cambio estratégico en la gestión de la base de datos: pasamos de una conexión tradicional con MySQL a la creación de un archivo .db mediante SQLite. Esta transición nos permite encapsular toda nuestra infraestructura de datos en un único archivo físico, eliminando la necesidad de configurar servidores de bases de datos externos y asegurando que el proceso de carga y consulta se complete sin errores de conexión o permisos en los equipos de la facultad.

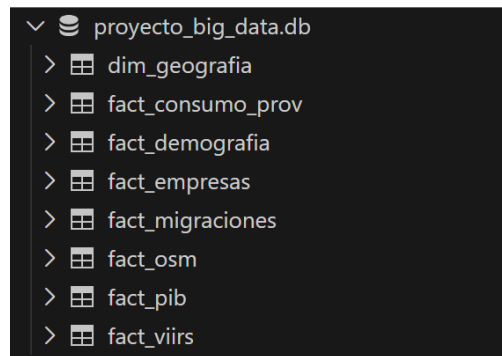
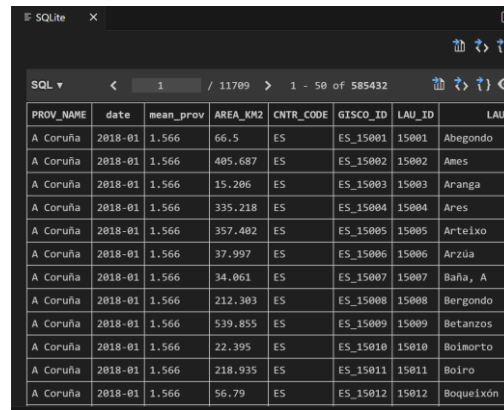


Ilustración 14-Dimensiones de la Tabla de Hechos

Para la estructura interna de esta base de datos, hemos optado por implementar un Modelo de Estrella, organizando la información en diversas Tablas de Hechos (Fact Tables) conectadas a una Dimensión Geográfica maestra. Esta elección es fundamental, ya que nuestro objetivo no es solo construir una aplicación web transaccional, sino crear un entorno de Big Data y análisis analítico eficiente para distintos ámbitos. Al tratar cada uno de nuestros siete archivos CSV de datos como tablas de hechos independientes, priorizamos la velocidad de lectura y la capacidad de realizar agregaciones masivas sobre la rigidez de una base de datos relacional tradicional. En este modelo, la redundancia controlada se convierte en una ventaja competitiva: nos permite cruzar métricas aparentemente inconexas, como la intensidad de la luz nocturna de VIIRS con el número de empresas de transporte, de forma casi instantánea.



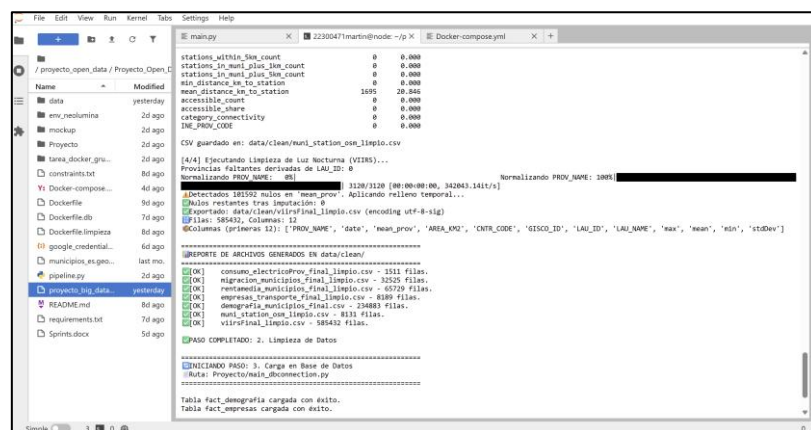
PROV_NAME	date	mean_prov	AREA_KM2	CNTR_CODE	GISCO_ID	LAU_ID	LAU_N
A Coruña	2018-01	1.566	66.5	ES	ES_15001	15001	Abegondo
A Coruña	2018-01	1.566	405.687	ES	ES_15002	15002	Ames
A Coruña	2018-01	1.566	15.206	ES	ES_15003	15003	Aranga
A Coruña	2018-01	1.566	335.218	ES	ES_15004	15004	Ares
A Coruña	2018-01	1.566	357.402	ES	ES_15005	15005	Arteixo
A Coruña	2018-01	1.566	37.997	ES	ES_15006	15006	Arzúa
A Coruña	2018-01	1.566	34.061	ES	ES_15007	15007	Baña, A
A Coruña	2018-01	1.566	212.303	ES	ES_15008	15008	Bergondo
A Coruña	2018-01	1.566	539.855	ES	ES_15009	15009	Betanzos
A Coruña	2018-01	1.566	22.395	ES	ES_15010	15010	Boimorto
A Coruña	2018-01	1.566	218.935	ES	ES_15011	15011	Boiro
A Coruña	2018-01	1.566	56.79	ES	ES_15012	15012	Boqueixón

Ilustración 15-Vista de datos viirs dentro de VS code

Para que esta arquitectura funcione, hemos desarrollado un flujo de integración en Python. Uno de los mayores retos fue garantizar que la unión de datos entre tablas fuera perfecta, por lo que implementamos un proceso de normalización de texto avanzado mediante una función. Este proceso nos permitió generar columnas de unión (muni_id_join y prov_id_join) en todos los datasets, asegurando que la integración sea robusta y libre de errores de codificación.

Finalmente, esta arquitectura nos otorga una escalabilidad de cara al futuro. Hemos diseñado el sistema de tal manera que la Tabla Maestra Geográfica, creada a partir de los datos de OSM por su exhaustividad, actúe como el pilar de todo el proyecto. Si en el futuro necesitáramos añadir nuevas fuentes de datos abiertos, solo tendríamos que generar una nueva fact table y conectarla mediante la columna id_join. Este diseño es vital para la fase de análisis, ya que nos permite subir o bajar el nivel de detalle, desde un municipio específico hasta una provincia completa, simplemente agrupando los hechos.

Como complemento necesario para garantizar la ejecución en el entorno de la universidad, tuvimos que solventar una limitación técnica importante: el ordenador L.O.R.C.A. no dispone de Docker instalado. Ante la imposibilidad de utilizar docker-compose para orquestar nuestros contenedores en ese equipo, desarrollamos un script adicional denominado pipeline.py. Este archivo actúa como un orquestador nativo en Python que ejecuta de forma secuencial y automatizada los tres procesos principales, asegurando que todo el flujo de datos se complete correctamente sin depender de la virtualización, pero manteniendo la misma lógica de modularidad que diseñamos para la versión de contenedor.



```

File Edit View Run Kernel Tabs Settings Help
main.py 2230471martin@node - /p X Docker-compose.yml X +

Name Modified
data yesterday
emr_neolumina 2d ago
mockup 2d ago
Proyecto 2d ago
bana_docker_gin... 2d ago
convertinet 8d ago
Yi Docker-compose... 4d ago
Dockerfile 9d ago
Dockerfileb 7d ago
Dockerfilelimpiez... 8d ago
google_credentia... 6d ago
municipios_es_ges... last mo.
pipeline.py 2d ago
proyecto_big_data... yesterday
README.md 8d ago
requirements.txt 7d ago
Sprints.docx 5d ago

stations_within_5km_count 0 0.000
stations_in_muni_plus_5km_count 0 0.000
stations_in_muni_plus_5km_count 0 0.000
min_distance_to_station 3895 10.046
accessible_count 0 0.000
accessible_share 0 0.000
category_connectivity 0 0.000
INE_PROV_CODE 0 0.000

CSV guardado en: data/clean/muni_station_clean.csv

[4/4] Ejecutando limpieza de Luz Nacturna (VIIRS)...
Provincias faltantes derivadas de LAU_ID: 0
Normalizando PROV_NAME: 0/1

Detectados 18152 milos en 'mean_prov'. Aplicando relleno temporal...
Datos restantes tras limpieza: 8
Exportado: data/clean/viirsfinal_limpio.csv (encoding utf-8-sig)
[131]: 585432, Columnas: 12
Columnas (primeras 12): ['PROV_NAME', 'date', 'mean_prov', 'AREA_KM2', 'CNTR_CODE', 'GISCO_ID', 'LAU_ID', 'LAU_NAME', 'max', 'min', 'stddev']

=====
REPORTE DE ARCHIVOS GENERADOS EN data/clean/
[OK] consumo_electricidad_prov_final_limpio.csv - 1511 filas.
[OK] migracion_municipio_final_limpio.csv - 32525 filas.
[OK] rentamedia_municipio_final_limpio.csv - 6729 filas.
[OK] empresas_transporte_final_limpio.csv - 8989 filas.
[OK] demografia_municipio_final_limpio.csv - 234883 filas.
[OK] muni_station_clean_limpio.csv - 8333 filas.
[OK] viirsfinal_limpio.csv - 585432 filas.

=====
ASO COMPLETADO: 2. Limpieza de Datos

=====
INICIANDO PASO: 3. Carga en Base de Datos
Ruta: Proyecto/main_dbconnection.py

Tabla fact_demografia cargada con éxito.
Tabla fact_empresa cargada con éxito.
  
```

Ilustración 16- Ejecucion Exitosa del Pipeline

Conclusiones y Trabajo a futuro

La finalización de esta primera etapa del proyecto ha permitido establecer la infraestructura necesaria para la gestión y el procesamiento de grandes volúmenes de datos procedentes de fuentes heterogéneas. Se ha desarrollado un flujo de trabajo que abarca desde la extracción automatizada de imágenes satelitales Sentinel-2 y datos socioeconómicos del INE, hasta su limpieza y posterior almacenamiento en una base de datos relacional. El trabajo se ha centrado en garantizar que los datos de diversas procedencias, como el transporte, la demografía y las luces nocturnas, queden correctamente vinculados mediante procesos de normalización. Esta fase concluye con un sistema funcional que permite ejecutar el pipeline de datos de manera secuencial, asegurando que la información esté preparada y verificada para las fases de análisis avanzado.

De cara a las fases de Big Data II y III, el proyecto se orientará hacia el análisis de datos, la implementación de modelos de Machine Learning (ML) y la visualización interactiva. En la segunda etapa, se procederá al entrenamiento de modelos con las imágenes descargadas y los datos limpios para identificar correlaciones y realizar predicciones basadas en la información obtenida. Se estudiará cómo analizar estos datos masivos para extraer patrones de valor que permitan entender mejor el territorio. Finalmente, en la tercera etapa, todos los resultados y procesos se integrarán en un mapa interactivo. Esta herramienta permitirá visualizar de forma dinámica tanto las capas de datos originales como los resultados de los modelos, facilitando una interpretación visual y accesible de toda la información procesada durante el desarrollo del proyecto.